

PRACTICA 4: PROCESAMIENTO E INTERPOLACION DE IMÁGENES TÁCTILES PARA UN GRIPPER ROBOTICO UTILIZANDO C++ Y PYTHON



Asignatura:
Sistemas Operativos

Curso:
2025-2026

Autores:
Lucia Castellanos Paz
Ander Zuazquita Pastor

1.Introducción

La percepción táctil es uno de los elementos mas importantes en los sistemas robóticos modernos. Este tipo de percepción es importante en aplicaciones de manipulación robótica, en la que los grippers deben interactuar con distintos objetos.

En esta practica vamos a realizar un sistema distribuido en C++ y Python capaz de procesar información táctil proveniente de un sensor robótico, aumentando su resolución mediante interpolación y generando representaciones visuales de la presión detectada.

A lo largo de la practica vamos a usar varios conceptos como el procesamiento de datos, la comunicación cliente-servidor mediante HTTP e intercambio de información entre aplicaciones de distintos lenguajes de programación.

2.Explicación del sistema

La aplicación sigue una arquitectura cliente-servidor.

El cliente desarrollado en C++ realiza las siguientes tareas:

- Lectura del archivo JSON
- Validacion de matrices 16x16
- Aplicacion de interpolación bilineal
- Envio de datos mediante HTTP POST

El servidor Python:

- Recibe las matrices interpoladas
- Reconstruye la información recibida
- Genera mapas de calor mediante Matplotlib
- Guarda automáticamente las imágenes generadas

El flujo del sistema permite transformar las capturas táctiles de baja resolución en representaciones visuales de mayor calidad.

3.Interpolación bilineal

La interpolación bilineal es una técnica usada para aumentar la resolución de imágenes y matrices bidimensionales.

En esta practica la usamos para convertir matrices de 16x16 en matrices de 128x128 usando un factor de escala de 8. Para eso se calculan valores intermedios a partir de los cuatros vecinos mas cercanos de cada posición.

En este procedimiento se generan transiciones suaves y mejora la calidad visual de los datos sin alterar la información original

4.Comunicación cliente-servidor

La comunicación entre aplicaciones se realiza mediante peticiones HTTP POST.

Cada matriz interpolada se convierte en formato JSON y se envía al servidor usando una estructura similar a esta:

```
{  
    "capture_id": 0,  
    "width": 128,  
    "height": 128,  
    "data": [[...]]  
}
```

El servidor recibe la información, reconstruye la matriz y genera una imagen táctil utilizando la librería Matplotlib

5.Resultados,ejecución y problemas encontrados

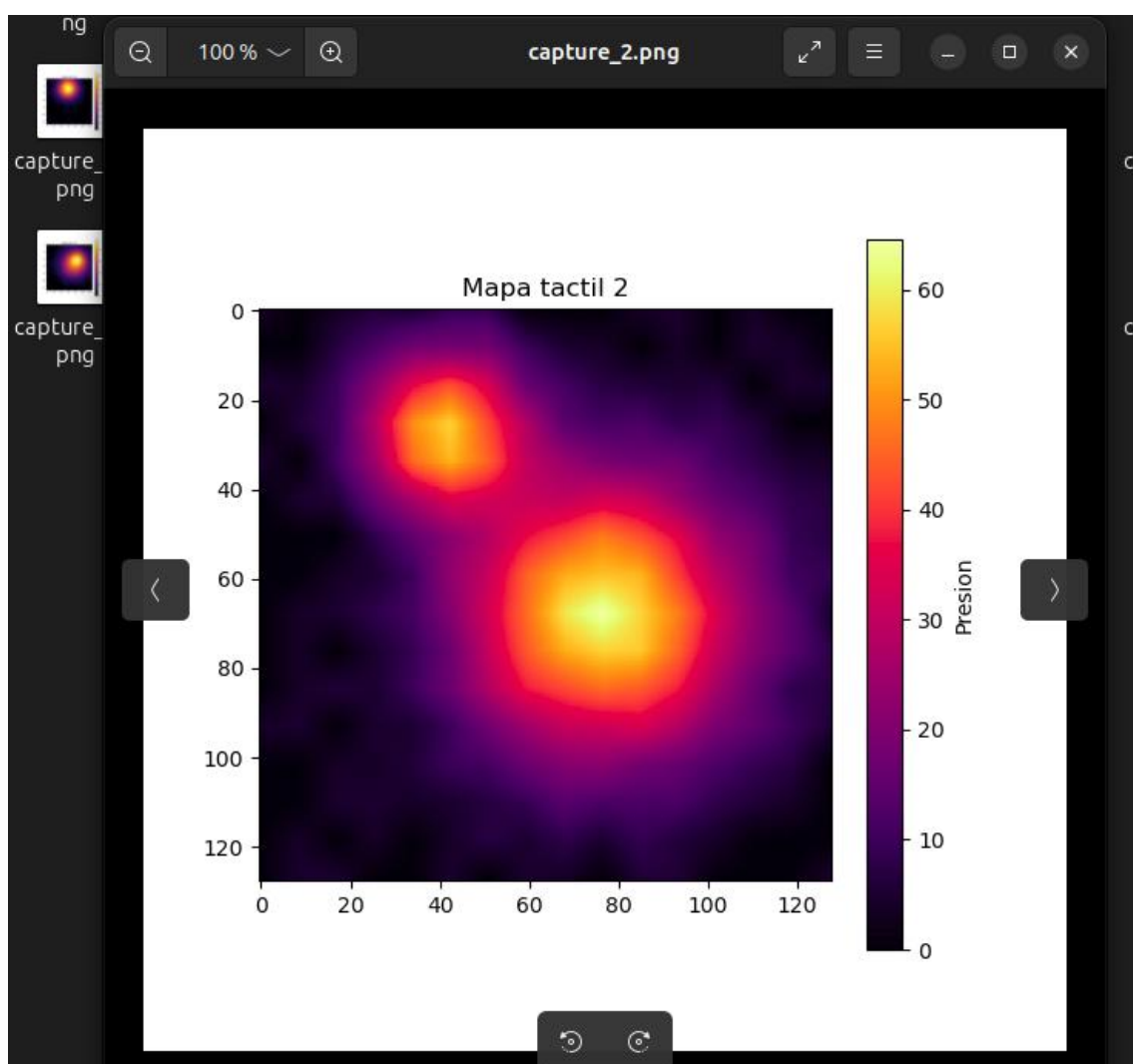
El sistema permite procesar de forma correcta las capturas táctiles almacenadas en el archivo JSON y generar imágenes de alta resolución que representan la distribución de presión detectada por el sensor.

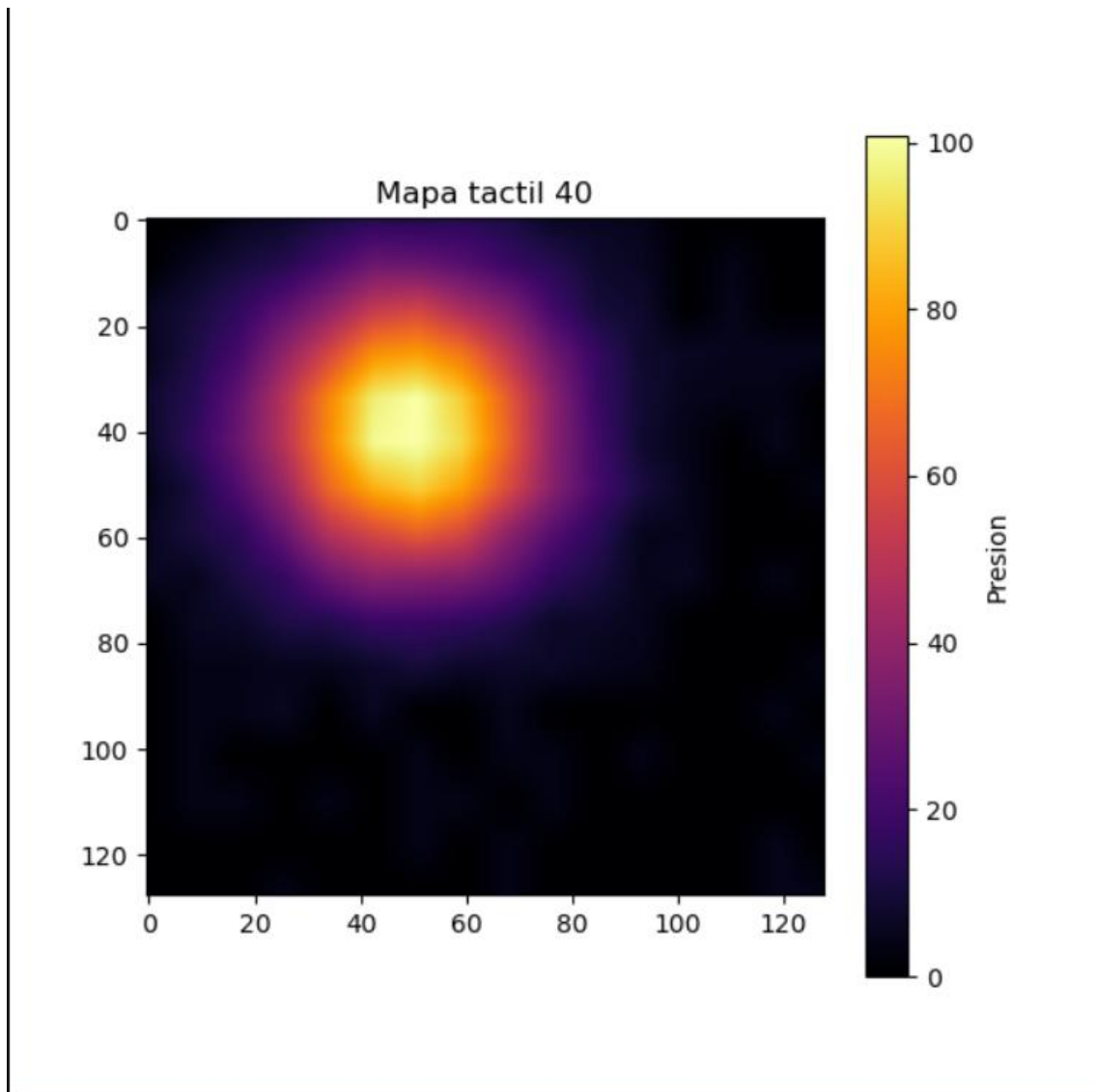
Capturas de pantalla de la ejecución;

```
anderzp in @anderHP in :~/Escritorio/practica4-so $ python3 python_server/server.py  
Servidor iniciado  
* Serving Flask app 'server'  
* Debug mode: on  
WARNING: This is a development server. Do not use it in a production deployment. Use a production  
WSGI server instead.  
* Running on all addresses (0.0.0.0)  
* Running on http://127.0.0.1:5000  
* Running on http://192.168.1.104:5000  
Press CTRL+C to quit  
* Restarting with watchdog (inotify)  
Servidor iniciado  
* Debugger is active!  
* Debugger PIN: 106-807-609  
1 app = Flask(__name__)  
2 # Crear carpeta para guardar imagenes  
3 if not os.path.exists("imagenes_generadas"):  
4     os.makedirs("imagenes_generadas")  
5  
6 # Ruta que recibira las matrices desde C++  
7 @app.route("/upload", methods=["POST"])  
8 def recibirMatriz():  
9     # Obtener datos enviados  
10     datos = request.json  
11  
12     # Leer informacion recibida  
13     idCaptura = datos["capture_id"]
```

```
~/Escritorio/practica4-so
anderzp in @anderHP in :~/Escritorio/practica4-so $ ./practica4
Capturas leídas: 50
{
  "mensaje": "Imagen generada correctamente"
}
Captura 0 enviada correctamente
{
  "mensaje": "Imagen generada correctamente"
}
Captura 1 enviada correctamente
{
  "mensaje": "Imagen generada correctamente"
}
Captura 2 enviada correctamente
{
  "mensaje": "Imagen generada correctamente"
}
Captura 3 enviada correctamente
{
  "mensaje": "Imagen generada correctamente"
}
Captura 4 enviada correctamente
{
  "mensaje": "Imagen generada correctamente"
}
Captura 5 enviada correctamente
{
  "mensaje": "Imagen generada correctamente"
}
Captura 6 enviada correctamente
{
  "mensaje": "Imagen generada correctamente"
}
Captura 7 enviada correctamente
{
  "mensaje": "Imagen generada correctamente"
}
Captura 8 enviada correctamente
{
  "mensaje": "Imagen generada correctamente"
}
```

Imagenes generadas:





(mas imagenes subidas a github,en la carpeta generada al ejecutar el programa “imágenes_generadas”)

Durante la practica encontramos algunos problemas relacionados con la implementación de la interpolación bilineal, la gestión del formato JSON y la comunicación entre el cliente C++ y el servidor Python. Estos errores los solucionamos mediante pruebas progresivas de cada modulo.

6.Conclusiones

La practica nos ha permitido usar conocimientos relacionados con el procesamiento de matrices, la programación modular, la comunicación cliente-servidor y la visualización científica.

El uso de la interpolación bilineal es una técnica eficaz para la representación visual de los datos táctiles. Además el uso del C++ para el procesamiento y Python para la generación de imágenes nos ha dejado desarrollar soluciones sencillas y eficientes.